The renderer, one frame at a time

This is a walkthrough of Lantern's renderer for someone who wants to build one like it. It follows a single frame from the game to the screen, names the file each step lives in, and ends with what this renderer leaves out and where each missing feature would go.

Read `engine-structure.md` first for where the folders sit, and `application-and-game-loop.md` for when `Render` is called.

The shape in one picture

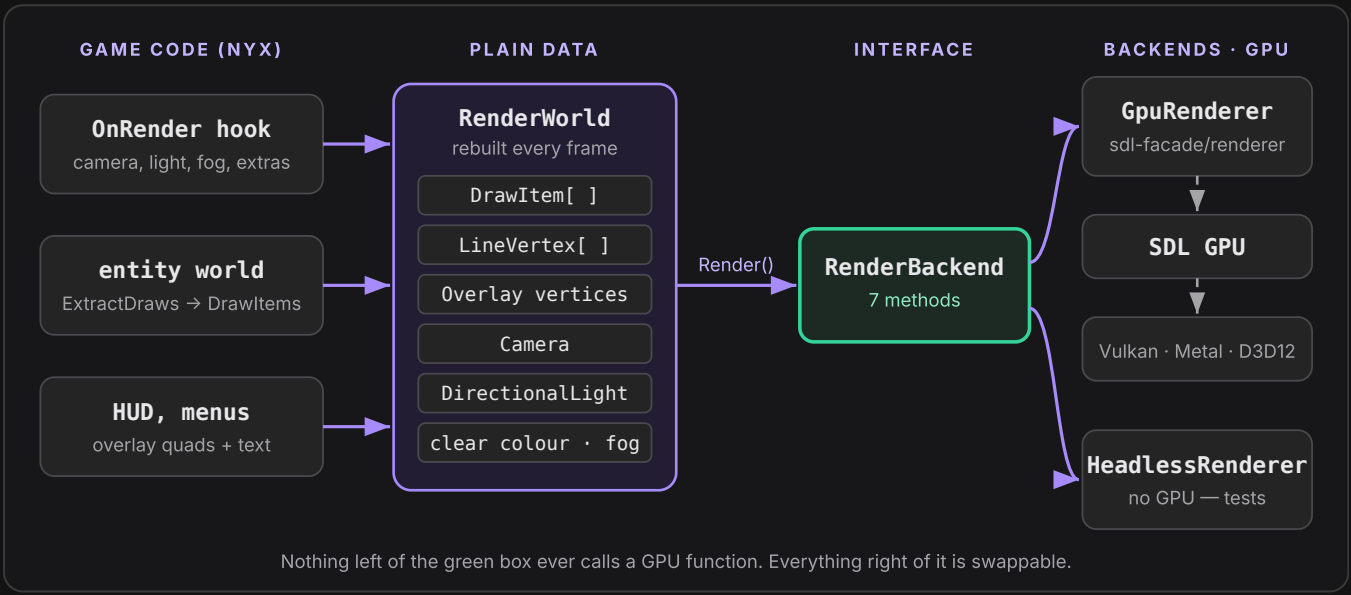


Figure 1 — The frame flows left to right as data. Only the backends know a GPU exists.

Two ideas carry the whole design:

1. **The frame is data.** Nothing above the renderer calls a GPU function. Each frame, the game fills a `RenderWorld`: a list of draw items, a list of debug lines, a list of 2D overlay vertices, and the camera, light, clear colour and fog. The renderer reads it and forgets it.
2. **One small interface between "what" and "how".** `RenderBackend` (`engine/src/renderer/RenderBackend.nox`) has seven methods: its driver name, create and remove a mesh, create and remove a texture, render a frame, and capture a frame to memory. `GpuRenderer` implements it over SDL GPU; `HeadlessRenderer` implements it with no GPU at all, for tests. A Vulkan or Direct3D renderer written by hand would be a third implementation, and nothing above the interface would change.

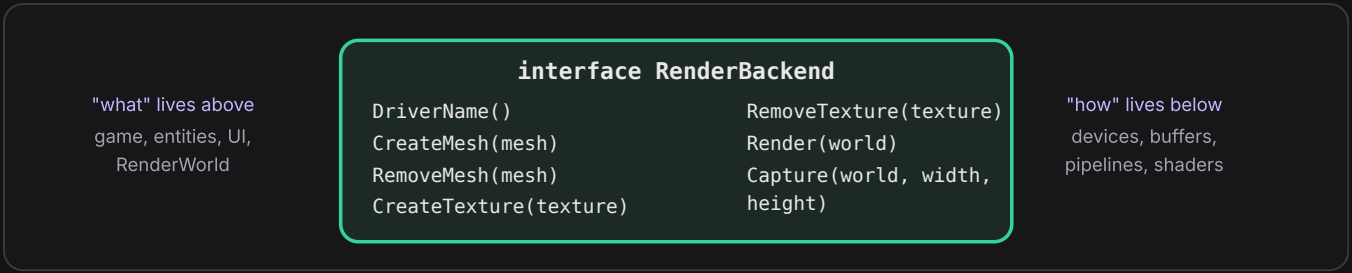


Figure 2 — The seven methods are the whole contract. Every method except `DriverName` returns a `Result` or nothing; see `RenderBackend.nox` for the full signatures.

The files

The renderer is split in two folders. `renderer/` holds platform-neutral data and maths; `sdl-facade/renderer/` holds everything that touches SDL GPU.

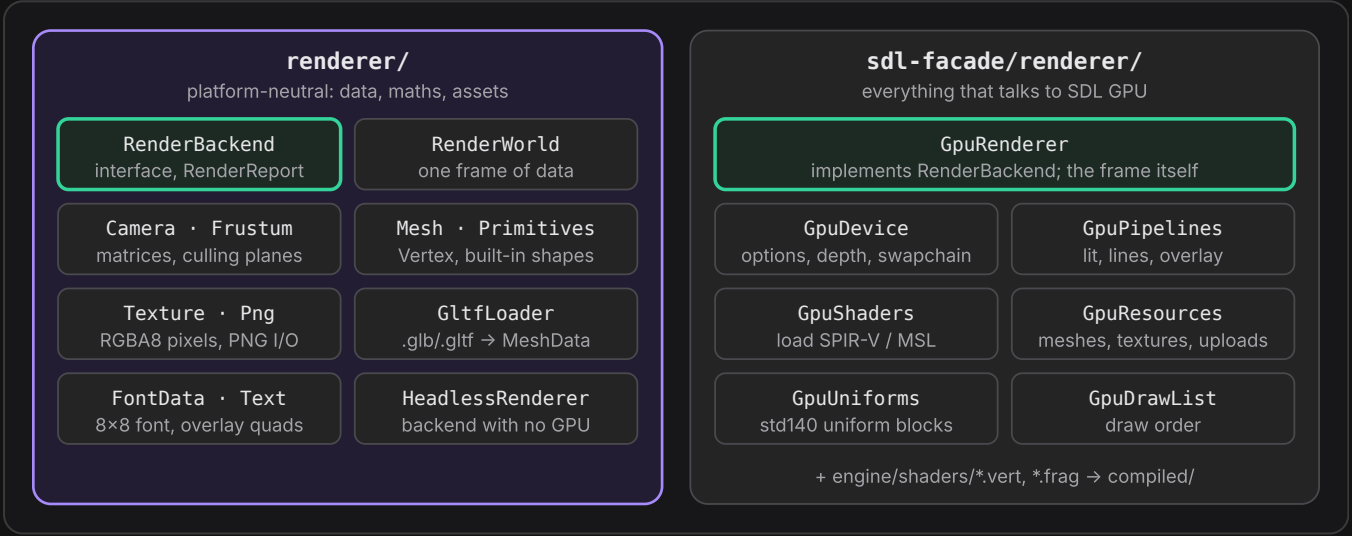


Figure 3 — The two halves of the renderer.

File	What it holds
`renderer/RenderBackend.nox`	The `RenderBackend` interface and `RenderReport`
`renderer/RenderWorld.nox`	`RenderWorld` (one frame), `DrawItem`, `DirectionalLight`, `LineVertex`, the 2D `Overlay`
`renderer/Camera.nox`	`Camera`: position, forward, field of view, near/far planes, view and projection matrices
`renderer/Frustum.nox`	The camera's view volume as six planes, and the box test used for culling
`renderer/Mesh.nox`	`Vertex` (48 bytes: position, normal, uv, colour) and `MeshData` (vertices and indices)
`renderer/Texture.nox`, `Png.nox`	`TextureData` (RGBA8 pixels, filter, wrap), PNG reading and writing
`renderer/Primitives.nox`	Cube, box, plane, ramp, capsule, sphere and gem meshes built in code
`renderer/GltfLoader.nox`	A `.glb` / `.gltf` file flattened into one `MeshData`
`renderer/FontData.nox`, `Text.nox`	An 8×8 bitmap font baked into a 128×48 atlas, and text laid out as overlay quads
`renderer/HeadlessRenderer.nox`	A `RenderBackend` with no GPU
`sdl-facade/renderer/GpuRenderer.nox`	`GpuRenderer`: the `RenderBackend` over SDL GPU; the frame itself
`sdl-facade/renderer/GpuDevice.nox`	Device options, the depth format and the swapchain set-up
`sdl-facade/renderer/GpuPipelines.nox`	The three pipelines: lit meshes, lines, overlay
`sdl-facade/renderer/GpuShaders.nox`	Finding and loading the compiled shaders (SPIR-V or MSL)
`sdl-facade/renderer/GpuResources.nox`	GPU meshes and textures, samplers, uploads, mipmaps, growable vertex buffers
`sdl-facade/renderer/GpuUniforms.nox`	The uniform blocks, laid out exactly as the shaders declare them
`sdl-facade/renderer/GpuDrawList.nox`	The order meshes are drawn in
`engine/shaders/*.vert`, `*.frag`	The GLSL shaders; `engine/shaders/compiled/` holds them built for Vulkan and Metal

1. Start-up: device, pipelines, default resources

`GpuRenderer:::Create(window, options)` (`GpuRenderer.nox`) does the one-time work, in five steps:

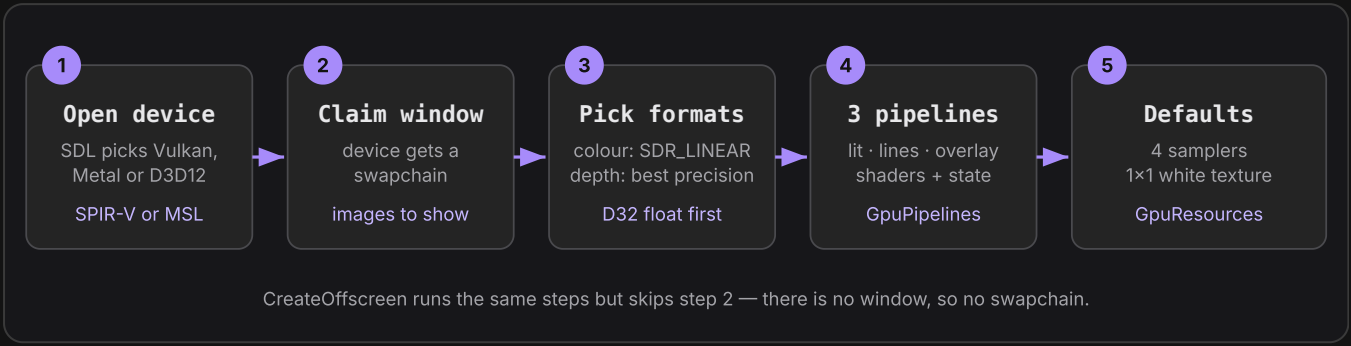


Figure 4 — Start-up, once per renderer.

1. **Open a device** (`OpenDevice`). SDL GPU chooses Vulkan, Metal or Direct3D 12 for the platform. The renderer asks for a device that accepts SPIR-V or MSL shaders, since those are the two compiled forms it ships.
2. **Claim the window** for the device, which gives it a swapchain: the images the window shows.
3. **Pick formats** (`GpuDevice.nox`). Colour comes from the swapchain. The swapchain is set to SDR_LINEAR where the window supports it, so the hardware converts linear colour to sRGB as pixels are written; otherwise the shaders do the conversion themselves. Depth takes the most precise format the card has (32-bit float first).
4. **Build three pipelines** (`GpuPipelines.nox`). A pipeline fixes everything about how one kind of geometry is drawn: its shaders, its vertex layout, the primitive type, culling, depth testing and blending.
5. **Make the samplers and the white texture** (`GpuResources.nox`). There are four samplers (linear or nearest, repeat or clamp) and every texture uses the one matching its settings. A 1×1 white texture stands in wherever a draw has no texture, so the shader can always sample one and multiply.

`GpuRenderer:::CreateOffscreen` does the same without a window; the editor's viewport and the GPU tests use it.

Why SDR_LINEAR matters

Lighting maths only works in *linear* colour, where doubling a number doubles the light. Screens expect *sRGB*, which spends more of its bits on dark tones. Someone has to convert between the two, exactly once:

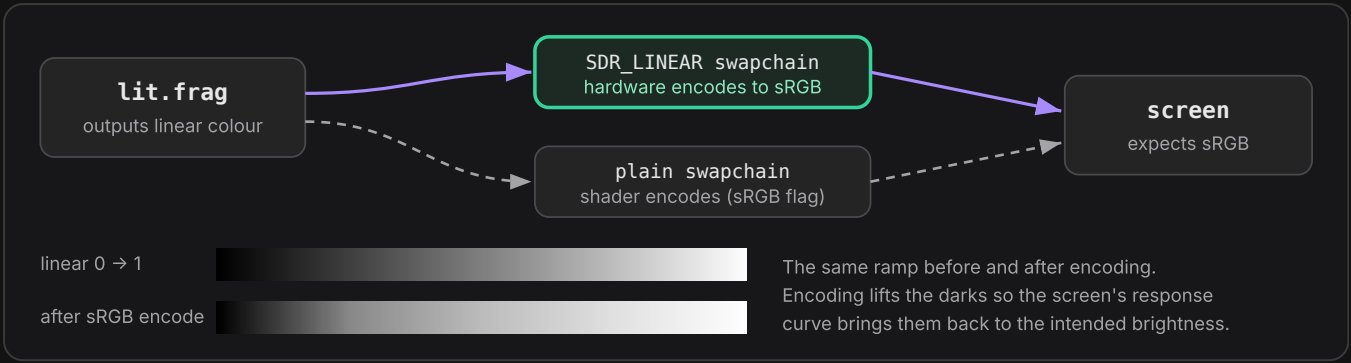


Figure 5 — Either the swapchain or the shader does the conversion. Never both, never neither.

The three pipelines

Pipeline	Draws	Vertex	Primitive	Back-face culling	Depth test / write	Blending
`lit`	meshes	48 bytes: position, normal, uv, colour	triangles	yes	yes / yes	off
`lines`	debug lines	28 bytes: position, colour	lines	no	yes / no	alpha
`overlay`	HUD, menus, text	32 bytes: 2D position, uv, colour	triangles	no	no / no	alpha

Each pipeline's shaders are loaded from `engine/shaders/compiled/` (`GpuShaders.nox`): `.spv` for Vulkan, `.msl` for Metal. Here is what each vertex looks like in memory, byte by byte:

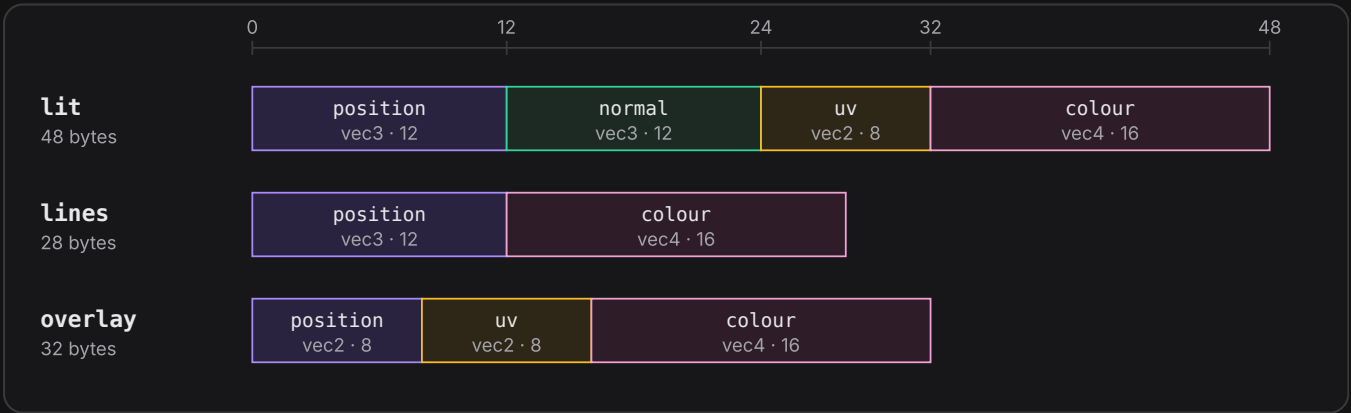


Figure 6 — Vertex layouts at 12 px per byte. Every field is `f32`: `Vertex`, `LineVertex` and `OverlayVertex`.

The pipeline settings follow from what each layer is for. Meshes are solid, so they cull back faces and write depth. Lines are thin and debug-only, so they test against mesh depth (a line behind a wall is hidden) but write none. The overlay is flat UI on top of everything, so it ignores depth altogether.

2. Long-lived resources: meshes and textures

A game uploads a mesh or texture once and refers to it by handle afterwards.

- `CreateMesh(MeshData)` validates the indices, creates a vertex buffer and an index buffer on the GPU, copies the data through a *transfer buffer* (memory the CPU can write and the GPU can read) in a copy pass, and keeps the mesh's bounding box for culling (`UploadMesh`).
- `CreateTexture(TextureData)` uploads the pixels the same way into an sRGB texture. A texture with linear filtering also gets a full mipmap chain generated on the GPU, so it doesn't shimmer when seen from far away (`UploadTexture`, `MipLevelCount`).

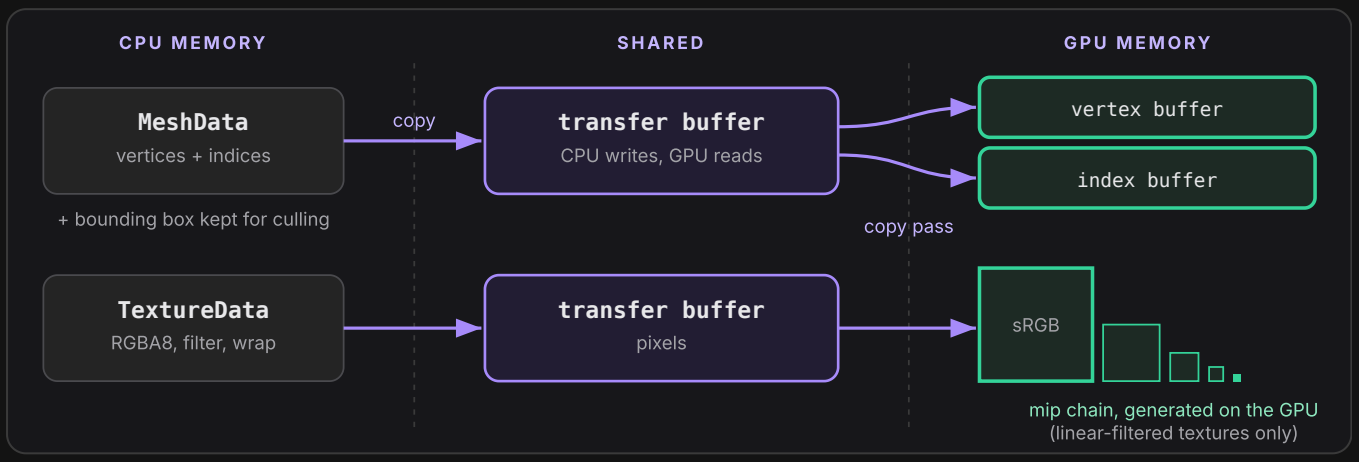


Figure 7 — Every upload goes CPU → transfer buffer → copy pass → GPU-only memory.

A mip chain is the same image at half size, quarter size and so on down to 1×1. When a texture covers only a few pixels on screen, the GPU samples a small mip instead of skipping across the full-size image, which is what causes shimmer.

- Both live in a `Pool`, which hands back a *generational handle*: an index and a generation number. Removing a mesh bumps its slot's generation, so an old handle stops matching. A draw that still holds it is skipped with a warning instead of reading freed memory (`core/Handle.nox`).



Figure 8 — A generational handle. Slot contents are illustrative. Generations start at 0, and a freed slot is reused later with its bumped generation, so a new handle never matches an old one.

3. The game fills a RenderWorld

Each frame, before anything touches the GPU, the game describes the frame (`host/GameHost.nox`, `Render`):

1. `renderWorld->Clear()` empties last frame's lists.
2. The game's `OnRender` hook sets the camera, light and fog, and may add draws, lines and overlay text directly.
3. The game's UI draws its boxes and text as overlay quads.
4. `ExtractDraws` (`engine/entities/World.nox`) walks every entity with a visible mesh renderer and adds a `DrawItem` for it: mesh handle, texture handle, world transform and tint colour. The transform is interpolated between the last two simulation steps with the loop's `alpha`, which is what keeps motion smooth when the display rate and the 60 Hz simulation don't line up (see `application-and-game-loop.md`).
5. The debug HUD adds its panel to the overlay.
6. `renderer->Render(renderWorld)`.

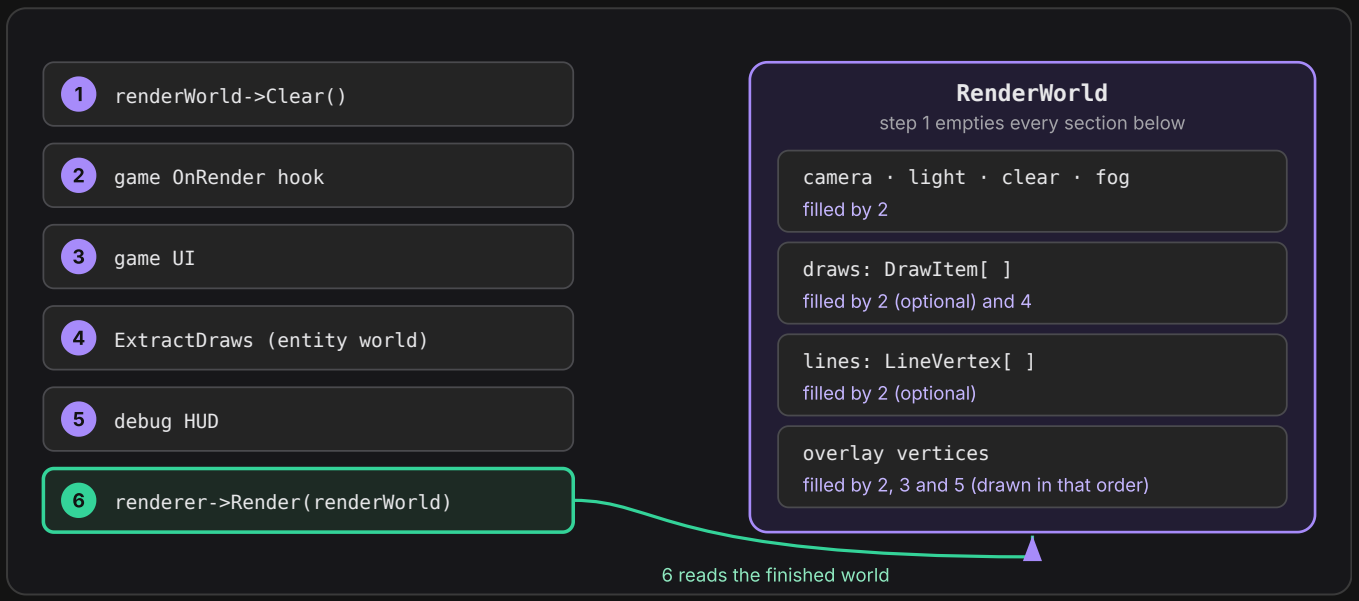


Figure 9 — Who writes which part of the frame. Overlay quads are drawn in the order they were added, so the HUD lands on top of the game UI.

Interpolation with `alpha`

The simulation ticks at a fixed 60 Hz; the display runs at whatever rate it runs. A frame usually lands *between* two simulation steps. Drawing the latest step as-is would make motion stutter, so `ExtractDraws` blends:

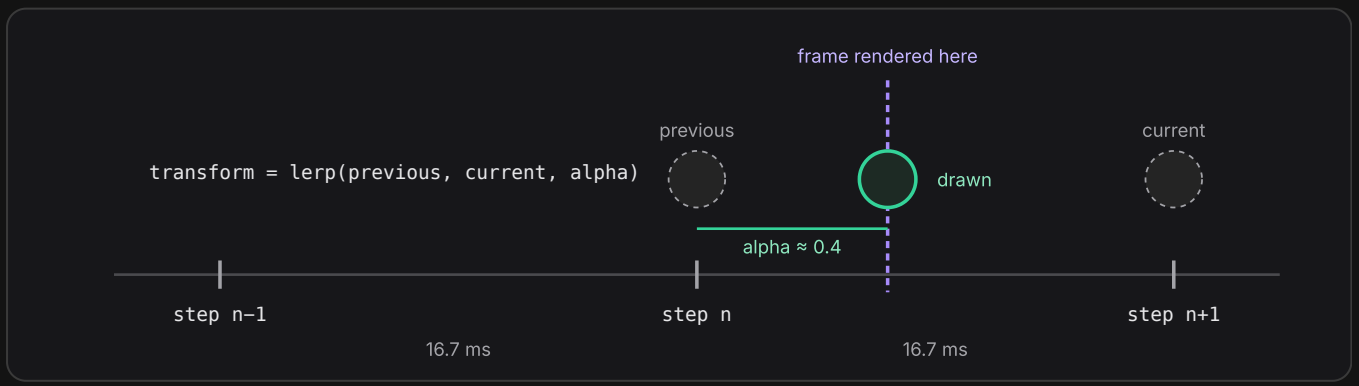


Figure 10 — The entity is drawn 40% of the way from its previous simulated position to its current one.

A `RenderWorld` holds only plain values and handles, never GPU objects. That is what lets the same frame go to `GpuRenderer`, `HeadlessRenderer` or `Capture` unchanged.

4. Render: one frame on the GPU

`GpuRenderer.Render` (`GpuRenderer.nox`):

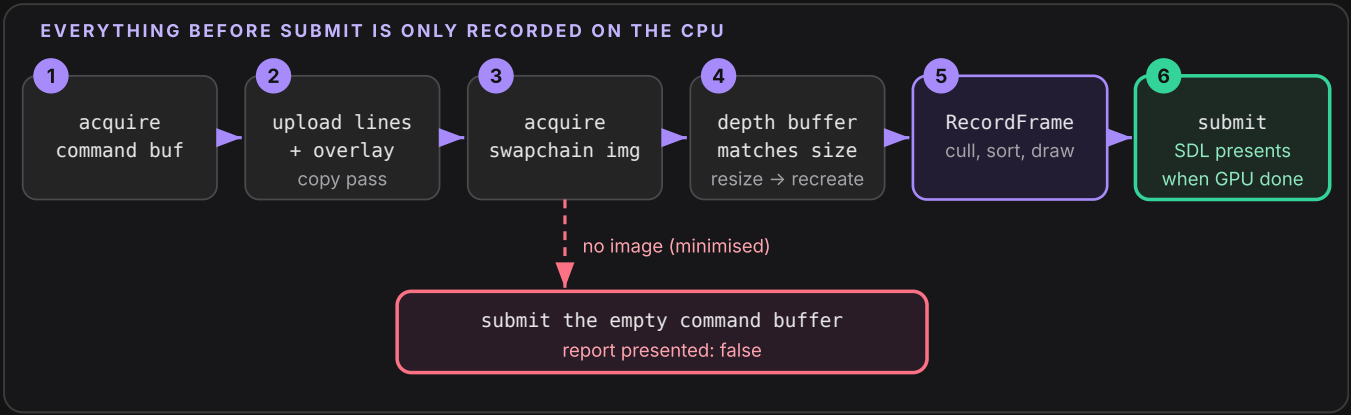


Figure 11 — Render(), with the early exit when there is nothing to draw into.

1. **Acquire a command buffer.** GPU work is recorded into a command buffer on the CPU and submitted all at once.
2. **Upload this frame's dynamic geometry** (`UploadDynamicGeometry`). Lines and overlay vertices change every frame, so they're copied into two growable vertex buffers in a copy pass. The buffers only grow, so a steady frame does no allocation.
3. **Acquire the swapchain image** to draw into. If there is none (the window is minimised), submit the empty command buffer and report `presented: false`.
4. **Make sure the depth buffer matches** the image's size; it's recreated when the window is resized (`EnsureDepthTarget`).
5. **Record the frame** (`RecordFrame`, below).
6. **Submit.** SDL presents the image once the GPU has finished with it.

RecordFrame

1. **Build the camera matrices.** The aspect ratio comes from the target's size. `Camera.ViewProjection` multiplies the projection (perspective, or orthographic for the editor's straight views) by the view (a right-handed look-to matrix). Lantern's axes: +Y up, the camera looks down -Z, and clip-space depth runs 0 to 1.



Figure 12 — Lantern's conventions. The camera's own position, forward and field of view live in `Camera.nox`.

2. **Cull** (`CollectVisibleDraws`). The view-projection matrix gives the frustum's six planes (`Frustum.nox`, the Gribb and Hartmann method). Each draw's mesh bounds are moved by its transform and tested against them: a box wholly outside any plane is skipped and counted in `drawsCulled`.

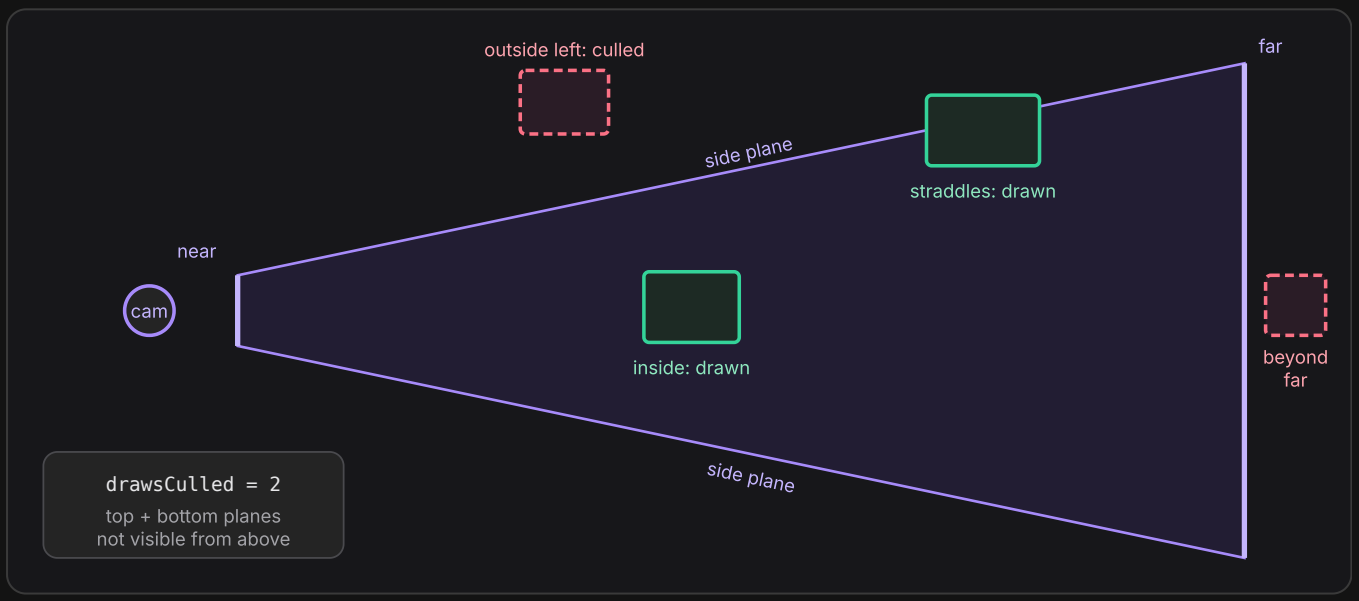


Figure 13 — The frustum seen from above. A box is culled only when it is wholly outside at least one plane; a box that straddles a plane is kept.

3. **Sort** (`GpuDrawList.nox`). The visible draws are sorted by texture, then mesh, then submission order, so consecutive draws share bindings.

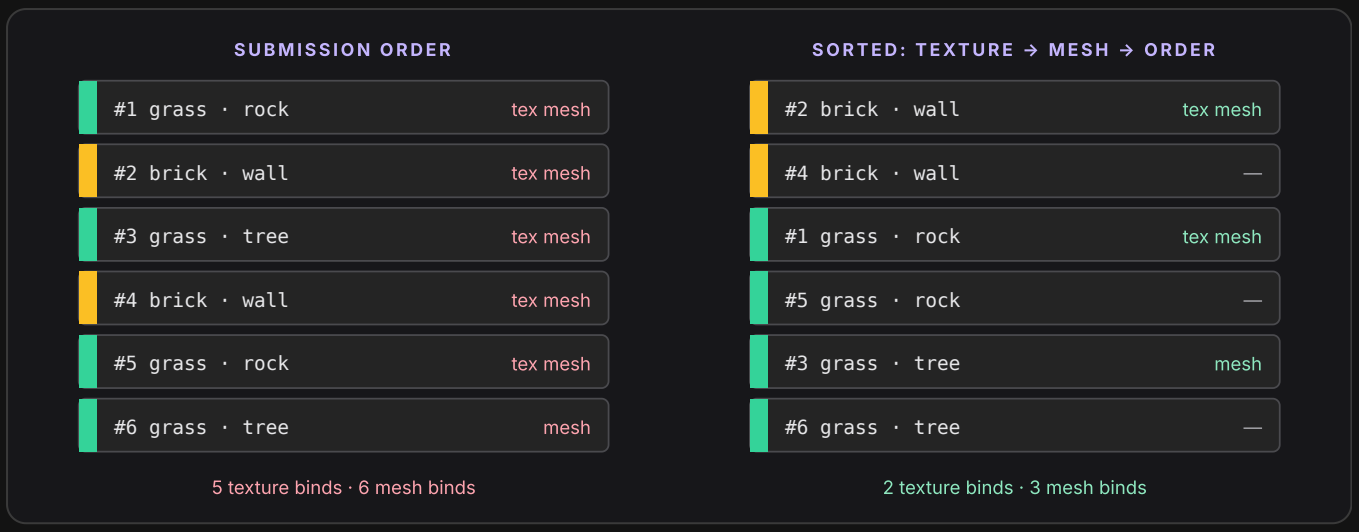


Figure 14 — Six draws, before and after sorting. Right-hand labels mark where a bind actually happens. Ties keep submission order (#2 before #4).

4. **Begin one render pass** with the colour image cleared to the world's clear colour and depth cleared to 1.0, the far plane.
5. **Draw the meshes** (`DrawMeshes`). Push the per-frame uniforms once: the view-projection matrix and camera position for the vertex shader, and the light, ambient and fog for the fragment shader. Then for each draw, bind its mesh's buffers and its texture only if they differ from the previous draw's, push the per-draw uniforms (the model matrix, its normal matrix and the tint) and issue one indexed draw.
6. **Draw the lines** (`DrawLines`), depth-tested against the meshes but not writing depth.
7. **Draw the overlay** (`DrawOverlay`) in screen pixels, with no depth test, so it sits on top. Every overlay vertex samples the font atlas; solid boxes use the atlas's white cell.
8. **End the pass.**

Steps 4 to 8 build the image in layers, inside a single render pass:

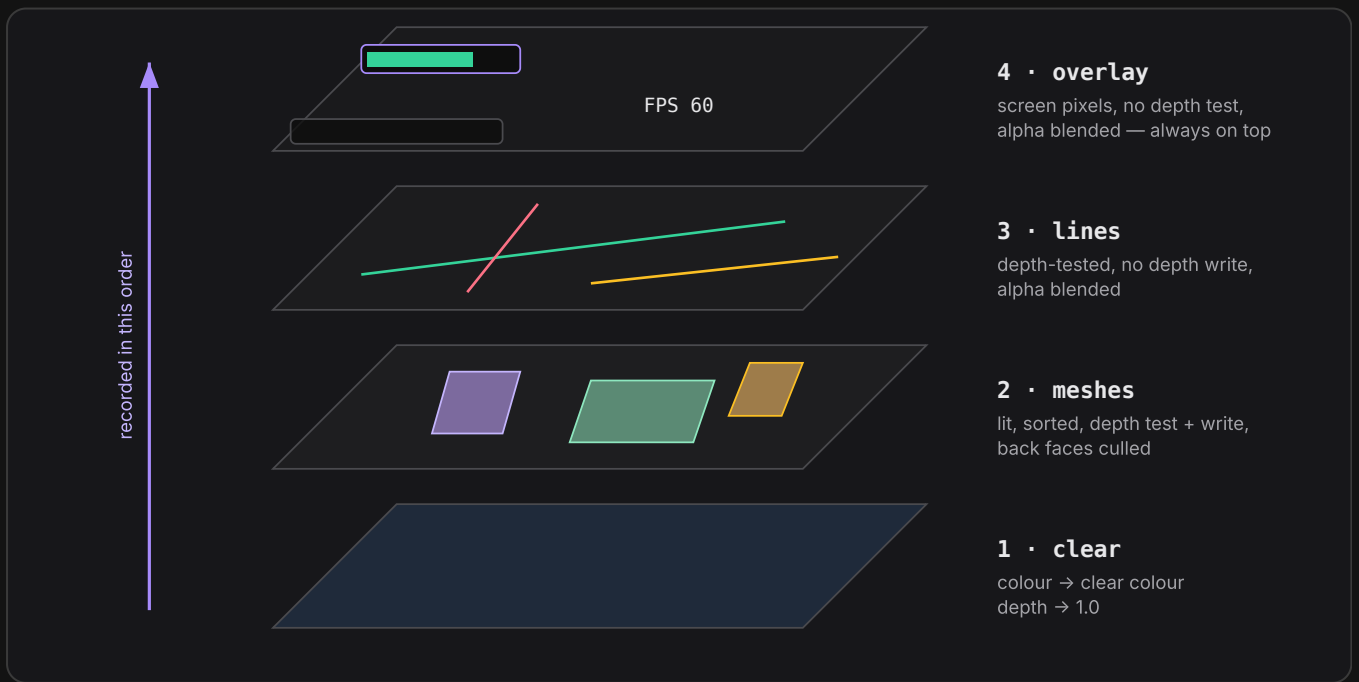


Figure 15 — One render pass, four layers. Each layer is drawn over the one beneath it.

The uniforms

Uniform blocks are small sets of constants pushed before a draw. `GpuUniforms.nox` lays each one out exactly as the GLSL declares it (std140, so every field is a whole number of 16-byte rows):

Stage, slot	Block	Pushed	Holds
vertex 0	`MatrixAndVectorUniforms` (80 bytes)	once per frame	view-projection, camera position
vertex 1	`ObjectUniforms` (128 bytes)	per draw	model matrix, normal matrix
fragment 0	`LightUniforms` (96 bytes)	once per frame	direction to the light, light and ambient colour, fog colour, fog range and the sRGB flag, camera position
fragment 1	`VectorUniforms` (16 bytes)	per draw	tint colour

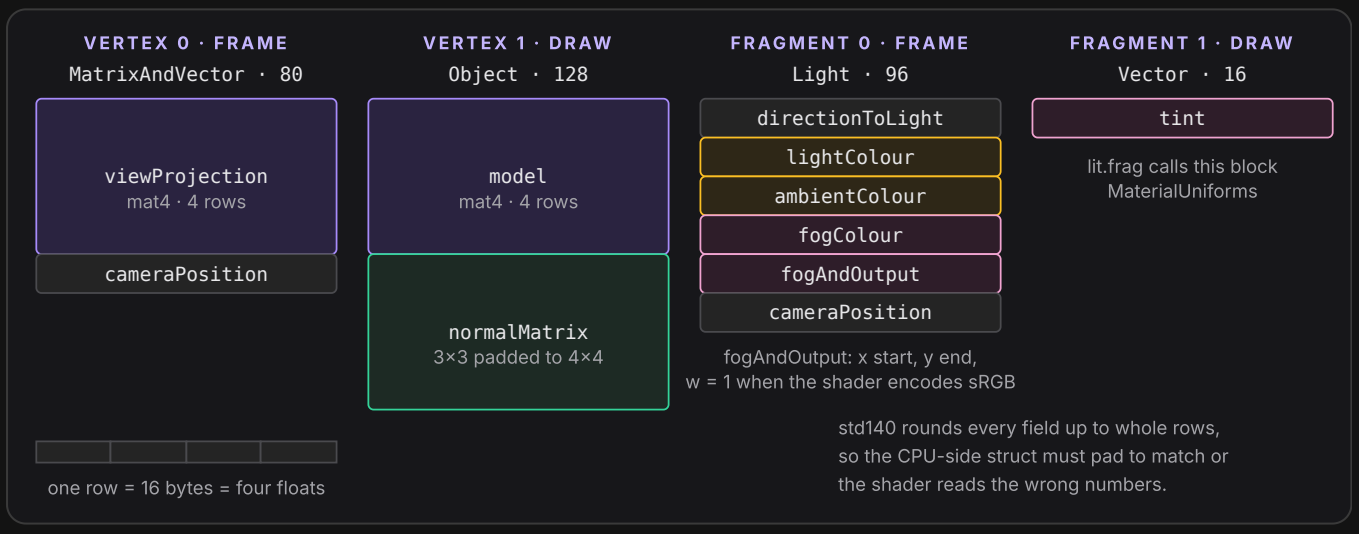


Figure 16 — The four uniform blocks drawn to scale, one row per 16 bytes. Field order matches `GpuUniforms.nox` and the GLSL blocks.

The normal matrix is the inverse transpose of the model matrix's 3×3 part. Normals moved by the model matrix itself would tilt off their surface under non-uniform scale.

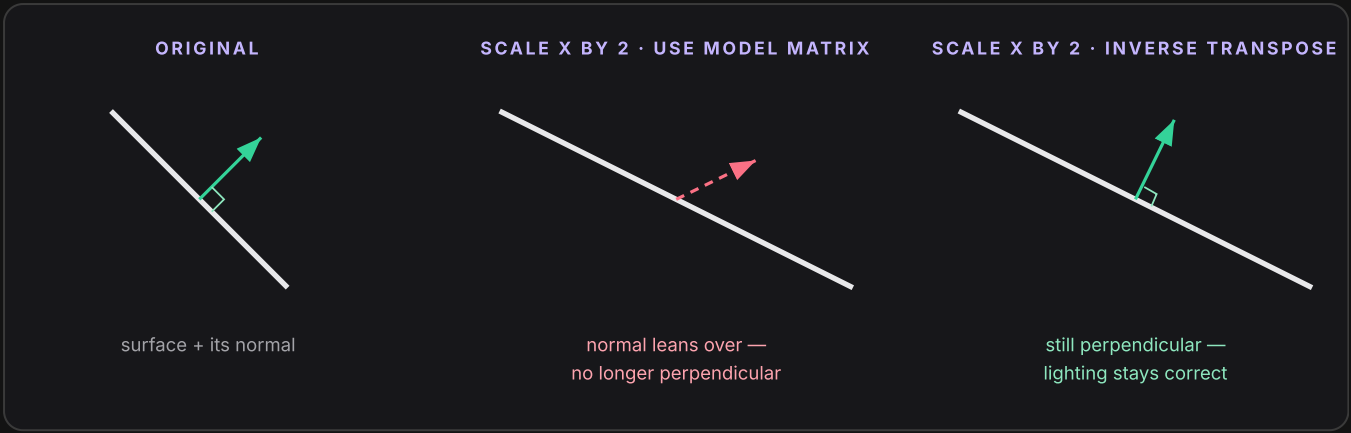


Figure 17 — Why the normal matrix exists. Stretching a surface sideways makes it shallower, so its normal must get steeper, not shallower.

The shaders

`lit.vert` moves each vertex to world space with the model matrix and to clip space with the view-projection matrix.

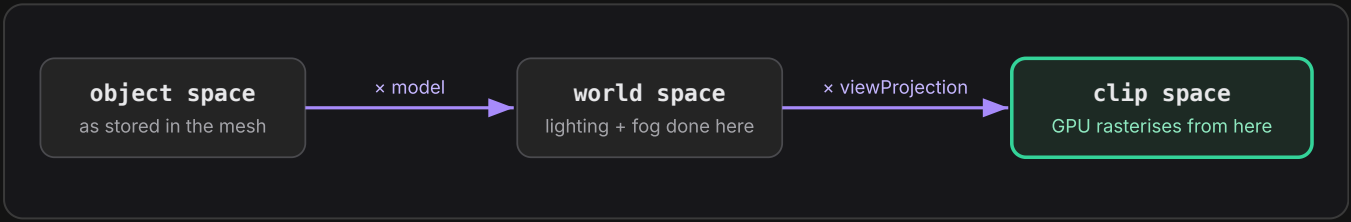


Figure 18 — The two matrix multiplies in `lit.vert`.

`lit.frag` computes:

```
albedo  = texture(uv) * vertex colour * tint
diffuse = max(dot(normal, directionToLight), 0)
sky     = 0.65 + 0.35 * normal.y                    a cheap hemisphere term: faces pointing
up get more ambient
colour  = albedo * (ambient * sky + lightColour * diffuse)
fog     = clamp((distanceToCamera - fogStart) / (fogEnd - fogStart), 0, 1)
colour  = mix(colour, fogColour, fog)               fog fades into the clear colour, hiding
the far plane
```

Each line of that has a simple picture:

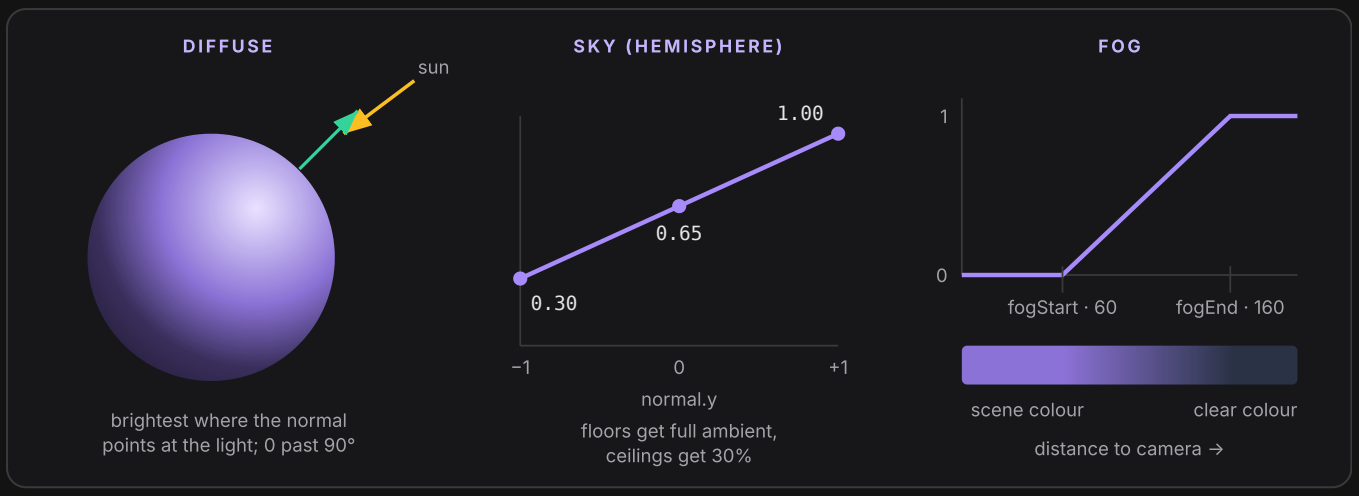


Figure 19 — Diffuse, the hemisphere ambient term, and linear fog. The renderer always fills the fog colour with the world's clear colour, so distant surfaces fade into the background before the far plane cuts them off. Defaults: fog from 60 to 160 units.

`lit.frag` also encodes to sRGB itself when the swapchain can't. `lines` passes colours through. `overlay.vert` turns pixel positions into clip space (`x / width * 2 - 1` , `1 - y / height * 2`).

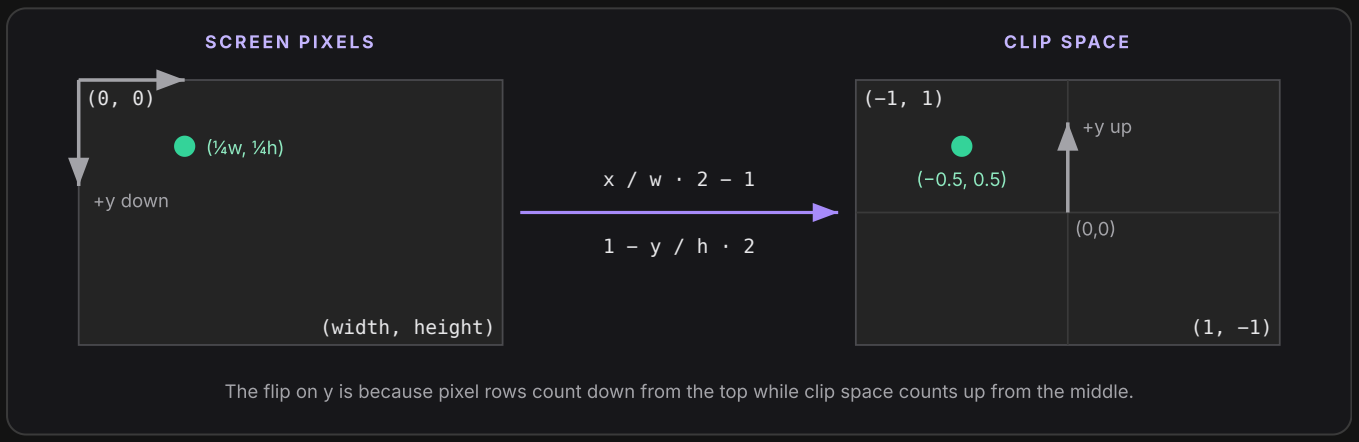


Figure 20 — The overlay's pixel-to-clip conversion, with one point traced through it.

5. Capture and off-screen rendering

`Capture(world, width, height)` records the same frame into a texture of its own instead of the swapchain, copies the texture into a download transfer buffer, waits on a fence for the GPU to finish, and copies the pixels back into a `TextureData` . On drivers whose images are blue-green-red, it swaps red and blue. Screenshots (`LANTERN_CAPTURE_PATH`) and the GPU tests use it. `RenderOffscreen` is the same with a target kept between frames, for the editor's viewport.

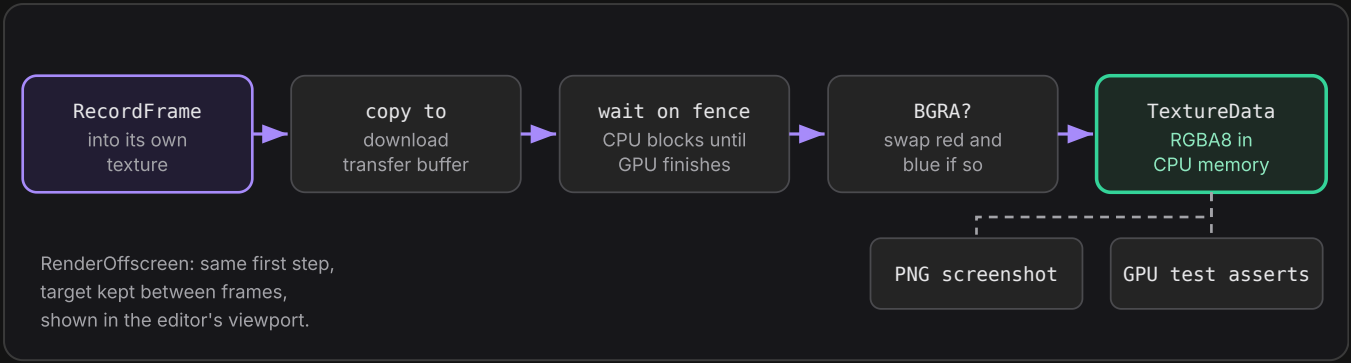


Figure 21 — Getting a frame back from the GPU. The fence wait is what makes this slow, and why games don't do it every frame.

6. Shutdown

`ReleaseDevice` waits for the GPU to go idle, then releases every mesh, texture, buffer, sampler and pipeline, gives the window back and destroys the device. It must run before the window is destroyed and before SDL shuts down, which is why the application deletes the game (which owns the renderer) before itself (see `application-and-game-loop.md`).

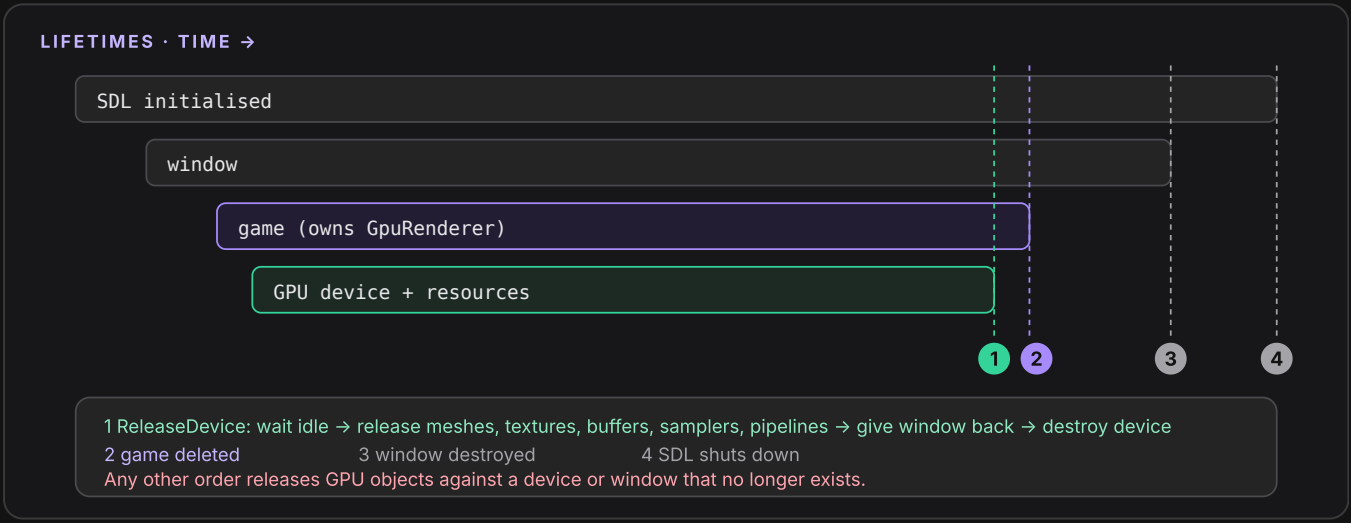


Figure 22 — Each lifetime nests inside the one above it, so teardown runs innermost first.

What this renderer leaves out, and where it would go

The renderer is small because it does one forward pass with one light, and SDL GPU handles the device, swapchain, memory and synchronisation underneath. These are the usual next features, and where each would fit:

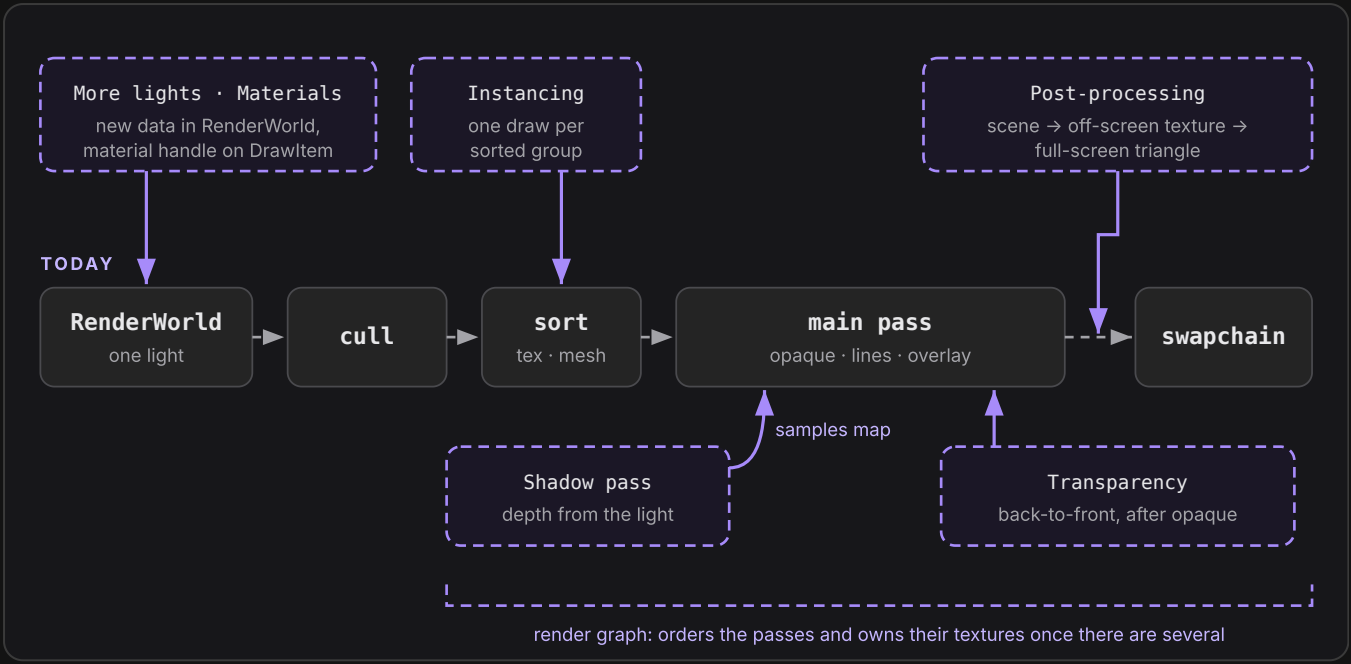


Figure 23 — Today's frame (grey) and where each missing feature plugs in (dashed purple).

Feature	What changes
Shadows from the sun	A depth-only pass from the light before the main pass, into a shadow-map texture; a fourth pipeline; the light's view-projection added to `LightUniforms`; `lit.frag` samples the map. `RecordFrame` grows a second pass.
Materials	A `Material` type in `renderer/` (shader, textures, parameters) and a material handle on `DrawItem` instead of one texture and a tint. The draw list sorts by pipeline, then material, then mesh.
More lights	An array of point and spot lights in `RenderWorld`, a storage buffer or larger uniform block, a loop in `lit.frag`; past a handful of lights, a clustered or deferred approach.
Transparency	A second list in `RenderWorld` for blended draws, sorted back to front and drawn after the opaque meshes with depth writes off.
Post-processing	Render the scene into an off-screen colour texture (as `Capture` does), then draw a full-screen triangle that samples it (tone mapping, bloom, anti-aliasing) into the swapchain.
Instancing	Group draws that share a mesh and material, put their transforms in a buffer, and issue one draw with an instance count. `GpuDrawList.nox` already sorts them together.
A render graph	Once there are several passes, describe each pass's inputs and outputs and let a small scheduler order them and manage their textures, instead of a hand-written `RecordFrame`.

Each of these lives behind `RenderBackend`: the game keeps filling a `RenderWorld`, and only `renderer/` (new vocabulary) and the backend (new GPU work) change.